End-to-End E-Commerce Analytics & Automation Platform

Project Documentation: Parts 1 to 3

1. Project Planning & Management
2. Literature Review
3. Requirements Gathering

Team

Member	Role
Ahmed Abdelkader	Team Leader | Supervision, Coordination, Data Sources & Extraction
Mariam Taleb	ETL & Data Cleaning
Aya Sharawy	Data Warehouse & SQL
Mahmoud Tarek	Analytics & Dashboard
Ismail Salah	Orchestration, Automation & Integration

Digital Egypt Pioneers Initiative (DEPI) | Graduation Project
Data Engineering Project | Version 1.4 | 19 October 2026
Track: Data Engineering | Instructor: Abdelrahman Ashraf

Contents

- **Part 1: Project Planning & Management**: 1.1 Project Proposal, 1.2 Project Plan, 1.3 Task Assignment & Roles, 1.4 Risk Assessment & Mitigation Plan, 1.5 KPIs
- **Part 2: Literature Review**: 2.1 Purpose and method, 2.2 Reviewed solutions, 2.3 Comparison, 2.4 Gap analysis, 2.5 ETL technology options, 2.6 Technical foundations, 2.7 Feedback & Evaluation, 2.8 Suggested Improvements, 2.9 Final Grading Criteria, 2.10 References
- **Part 3: Requirements Gathering**: 3.1 Key terms, 3.2 Stakeholders, 3.3 User stories and use cases, 3.4 Functional requirements, 3.5 Non-functional requirements, 3.6 Constraints, assumptions, and open decisions

Document control

Item	Detail
Project	End-to-End E-Commerce Analytics & Automation Platform
Document	Project Documentation, Parts 1 to 3 (Planning & Management, Literature Review, Requirements Gathering)
Version / date	1.4 / 19 October 2026
Submission	Monday, 19 October 2026 (official final deadline, not extendable). One PDF uploaded to the team GitHub repository; the Team Leader then submits it on the LMS as "Graduation Project".
Later DEPI dates	System Analysis & Design: 6 Nov 2026 | Implementation: 30 Nov 2026 | Final Presentation, Testing & Reports: 4 Dec 2026

Revision history

Version	Date	Changes
1.0	16 Oct 2026	First complete draft of Parts 1 to 3.
1.1	18 Oct 2026	No Python for ETL or automation (instructor rule). Stack: Apache Hop, PostgreSQL, Power BI, Windows Task Scheduler. Data from a public Kaggle dataset. Roles, risks, requirements, and timeline updated.
1.2	18 Oct 2026	Layout cleaned up so tables no longer split across pages. Part 3 redesigned: key terms, requirements grouped by pipeline stage, plain-language wording.
1.3	18 Oct 2026	Team leader also supervises and coordinates all streams, alongside data sources, extraction, and the README. Gantt chart updated.
1.4	19 Oct 2026	Deadline wording aligned with the official CLS announcement (19 Oct 2026, final). Added the Literature Review items required by the guidelines: Feedback & Evaluation, Suggested Improvements, Final Grading Criteria. Document control and table layout fixed.

Part 1: Project Planning & Management

1.1 Project Proposal

Overview

The End-to-End E-Commerce Analytics & Automation Platform is a data engineering project that turns raw e-commerce transactions into clean, structured data, customer loyalty insights, and automatically generated reports. For development and demonstration, the team uses a public e-commerce dataset from Kaggle. The data is extracted and cleaned with Apache Hop, stored in a PostgreSQL star schema, and analyzed with SQL. Loyalty points and tiers are calculated from editable rules, results appear in a Power BI dashboard, and weekly and monthly reports are produced by scheduled jobs. Following the instructor's rules, no Python is used for ETL or automation.

Problem statement

- E-commerce data is scattered across files and systems, so companies cannot easily see who their best customers are.
- Loyalty points and rewards are tracked manually or in spreadsheets, with no link to purchase history.
- Weekly and monthly reports are built by hand, which is slow and error-prone.

Objectives

- **O1.** Build repeatable pipelines in an ETL tool (Apache Hop) that extract, clean, and load e-commerce data without manual steps.
- **O2.** Store clean data in a PostgreSQL star schema that supports fast analytical queries.
- **O3.** Calculate loyalty points and tiers from purchase history using editable rules.
- **O4.** Present sales and loyalty KPIs on a Power BI dashboard with date filters.
- **O5.** Generate weekly and monthly reports automatically through scheduled jobs.

Scope

In scope	Out of scope
ETL pipelines in Apache Hop with validation and logging PostgreSQL star schema and SQL analytics Rule-based loyalty points, tiers, and RFM segmentation Power BI dashboard with DAX KPIs Scheduled weekly and monthly reports with Windows Task Scheduler Public Kaggle e-commerce data only	Python-based ETL or automation (instructor rule) Web scraping and live store APIs Real-time streaming data Payment processing and checkout features Predictive machine learning Real personal customer data

Expected outcomes

A documented GitHub repository, a Kaggle sample dataset with its source recorded, Apache Hop pipelines, a PostgreSQL schema with SQL queries, a Power BI dashboard with loyalty views, scheduled weekly and monthly reports, and a final report with a configuration guide.

1.2 Project Plan

The first three documentation parts are due on the official final deadline of **19 October 2026**, as announced by CLS Learning Solutions (final, not extendable). The remaining stages follow the DEPI schedule: System Analysis & Design on 6 November, Implementation on 30 November, and the final presentation, testing, and reports on 4 December 2026. Because the team is new to Apache Hop, PostgreSQL, and Power BI, a tool setup and training phase runs alongside System Analysis & Design.

Phase	Start	End	Main output
Project Planning & Management	2 Oct 2026	19 Oct 2026	Proposal, plan, roles, risks, KPIs
Literature Review	5 Oct 2026	19 Oct 2026	Comparison with similar solutions and tools, evaluation, improvements, grading criteria, references
Requirements Gathering	5 Oct 2026	19 Oct 2026	Stakeholders, user stories, requirements

Phase	Start	End	Main output
System Analysis & Design	19 Oct 2026	6 Nov 2026	Use cases, architecture, star schema, data flow, wireframes
Tool setup & training (all members)	19 Oct 2026	30 Oct 2026	Java, Hop, PostgreSQL, Power BI installed; first exercise done by every member
Implementation	6 Nov 2026	30 Nov 2026	Pipelines, database, dashboard, scheduling
Final Presentation, Testing & Reports	23 Nov 2026	4 Dec 2026	Test report, final report, presentation, project infographic

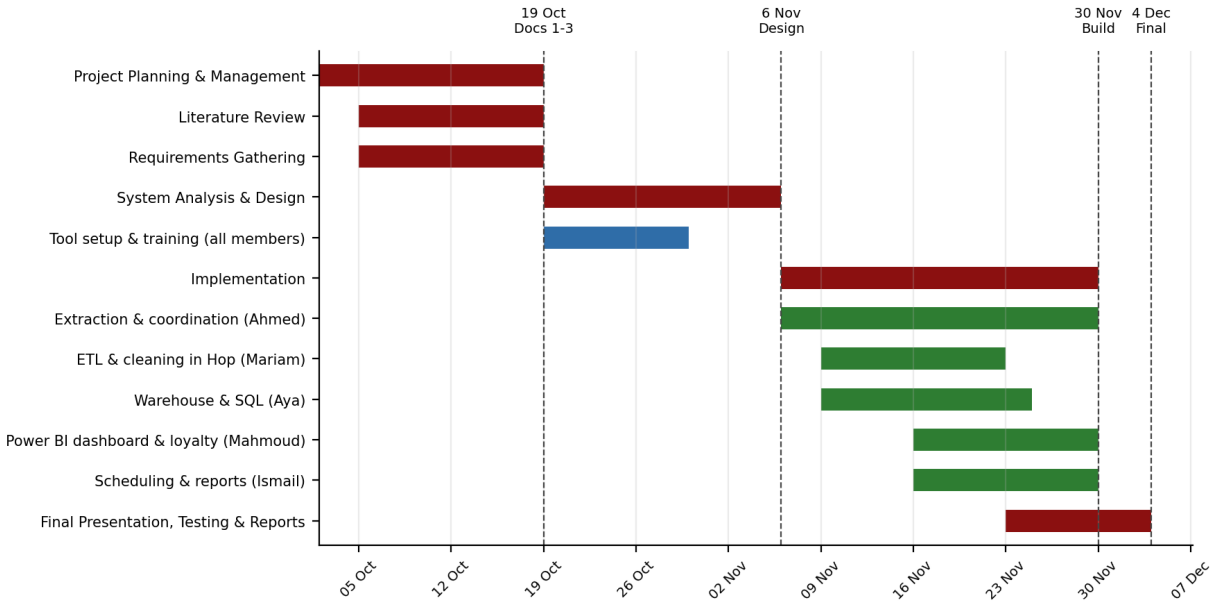


Figure 1. Project timeline (Gantt chart). Green bars show the five implementation streams, one per team member.

Milestones and deliverables

Date	Milestone	Deliverable
19 Oct 2026	Documentation Parts 1 to 3 submitted (official final deadline)	One PDF in the GitHub repository; Team Leader submits it on the LMS as "Graduation Project"
30 Oct 2026	Tools ready	Same Java and Hop versions installed by all; each member has run a first exercise
6 Nov 2026	System Analysis & Design complete	Use case diagram, architecture, star schema (ERD), data dictionary, wireframes
16 Nov 2026	Extraction working	Raw Kaggle files in the Landing Zone and loaded to staging
25 Nov 2026	Data warehouse loaded	Clean data in the star schema, SQL queries documented
30 Nov 2026	Implementation complete	Working pipelines, loyalty calculation, Power BI dashboard, scheduled reports
4 Dec 2026	Final delivery	Test report, final report, configuration guide, presentation, project infographic

Resource allocation

Resource	Allocation
People	Five team members, one stream each, with the team leader coordinating
ETL	Apache Hop (visual ETL tool); pipelines run from the command line with hop-run
Database	PostgreSQL with SQL
Analytics	Power BI Desktop with DAX
Scheduling	Windows Task Scheduler running Hop from the command line
Other tools	Postman (optional, for testing any API); Git and GitHub for version control
Data	A public Kaggle e-commerce dataset, downloaded manually; its name, link, and licence are recorded in the README
Environment	Windows laptops with the same Java and Apache Hop versions; one shared GitHub repository holding Hop pipeline and workflow files and SQL scripts

1.3 Task Assignment & Roles

Member	Role	Responsibilities	Deliverables
Ahmed Abdelkader (Team Leader)	Team Leader, Data Sources & Extraction	Supervise and follow up with every member; track milestones and risks; download the Kaggle dataset and record its source and licence; keep raw files in a Landing Zone; build Hop extraction pipelines into staging; own the repository, README, and final documentation	Landing Zone data, extraction pipelines, progress tracking, README
Mariam Taleb	ETL & Data Cleaning	Build cleaning pipelines in Apache Hop: handle missing values and duplicates; unify types and dates; apply validation rules; write rejected rows with a reason; deliver clean data for loading	Cleaning pipelines, clean data, rejected-rows output
Aya Sharawy	Data Warehouse & SQL	Design the PostgreSQL star schema with keys and relationships; load clean data; write SQL for revenue, top products and categories, loyalty points and tiers (from a rules table), and the weekly and monthly report queries	Schema, loading scripts, documented queries
Mahmoud Tarek	Analytics & Dashboard	Define KPIs; build the Power BI dashboard with DAX measures and loyalty views; design the report layout (exportable to PDF); check dashboard numbers against SQL results	Power BI dashboard and report layout
Ismail Salah	Orchestration, Automation & Integration	Chain the Hop pipelines and workflows in the right order; schedule them with Windows Task Scheduler; add error handling, logs, and run tests; run the report-generation jobs	Scheduled pipeline, logs, tests

Team working rules

- The first two members agree on file formats and column names early, so cleaning does not start on an unknown schema.
- The dashboard owner defines KPIs early, so the warehouse owner can prepare the matching queries.
- The automation owner takes part from the start, so every stage can be chained without rework.
- Everyone uses the same Java and Apache Hop versions, recorded in the README.
- Each member completes a first exercise in their tool by 30 October.
- Each member commits their own files to GitHub and merges gradually through pull requests.
- The team meets regularly to track progress and solve problems.

1.4 Risk Assessment & Mitigation Plan

ID	Risk	Prob.	Impact	Mitigation	Owner
R1	Kaggle data has missing IDs, cancelled orders, negative quantities, and text dates	High	Medium	Agree a data contract early; document a rule for each problem; reject or quarantine invalid rows with a reason	Mariam
R2	Schema changes after the ETL is built	Medium	High	Freeze the schema after System Analysis & Design (6 Nov); allow changes only through the team leader	Aya
R3	One member's part is late and blocks the next stage	Medium	High	Use small sample files so stages can be tested independently; hold regular check-ins	Ahmed
R4	Stages do not fit together (formats, column names)	Medium	High	Share one data dictionary and file formats; run an early integration test	Ismail
R5	Loyalty feature grows beyond scope	Medium	Medium	Limit version 1 to points, three tiers, and editable thresholds; keep extra ideas in a backlog	Ahmed
R6	Loyalty rules cannot be edited directly inside Power BI	Medium	Medium	Store rules in a table or file that a user edits and the pipeline reads; ask the instructor whether a richer interface is needed	Mahmoud
R7	Credentials pushed to GitHub	Low	High	Keep passwords in local configuration excluded by .gitignore; use public Kaggle data only	Ismail
R8	Team is new to Apache Hop, PostgreSQL, and Power BI	High	High	Setup and training phase (19 to 30 Oct); one first exercise per member; use official documentation; fall back to Spark SQL if Hop fails	All
R9	Instructor does not approve Apache Hop as the ETL tool	Medium	High	Confirm approval this week; fallback options are Spark SQL or Talend Studio if the instructor provides the installer	Ahmed
R10	Different Java or Hop versions on team laptops	Medium	Medium	Agree one version; record it in the README; test the pipeline on a second laptop	Ismail
R11	Time pressure from other courses and deadlines	Medium	Medium	Keep tasks small and a buffer in the final week	All
R12	Missing the final 19 Oct submission deadline (not extendable)	Low	High	Freeze content on 18 Oct; Team Leader uploads the PDF to GitHub and the LMS early on 19 Oct and confirms the upload	Ahmed

1.5 Key Performance Indicators (KPIs)

The KPIs below measure whether the project succeeds. Targets apply to the sample dataset and will be checked during testing.

KPI	Target	How it is measured
Tool readiness	Every member runs a first pipeline, query, or report in their tool by 30 Oct	Short demo to the team
Pipeline success rate	At least 95% of scheduled runs finish without manual fixes	Run logs
Data quality pass rate	At least 95% of sample rows pass validation; 100% of rejected rows have a logged reason	Hop logs and rejected-rows output
Report accuracy	100% of report totals match SQL query results	Comparison test per report
Report generation time	Weekly report under 2 minutes; monthly report under 5 minutes	Timed test runs
Dashboard response time	Each Power BI view loads in under 5 seconds	Timed test on sample data
Scheduler reliability (uptime)	Scheduled jobs start on time in at least 95% of test runs	Task Scheduler history and logs

KPI	Target	How it is measured
User adoption	At least 80% of test users change a loyalty rule and open a report without help	Short usability test with 5 users
Delivery	All planned deliverables submitted on the DEPI dates; all five members' work merged in one repository	Repository history and submission records

Part 2: Literature Review

2.1 Purpose and method

This review studies existing solutions that address parts of our problem: loyalty management, store reporting, and business intelligence. It also compares the ETL tools the team could use under the instructor's rules. We reviewed vendor pages, the Shopify Help Center, Microsoft and Apache documentation, and independent software review sites in October 2026. We recorded what each solution does well, where it falls short for our use case, and how our project differs. Features and prices change often, and several prices come from third-party listings, so they are indicative only.

2.2 Reviewed solutions

Smile.io (loyalty app)

Smile.io is a loyalty and rewards platform for e-commerce merchants that offers points programs, referral programs, and VIP membership tiers, mainly for Shopify and BigCommerce stores [2]. VIP tiers are available only on paid plans, and one review lists the first such plan at about $199 per month [2][3]. Higher plans give access to raw loyalty data for analysis in external BI tools [2].

Strengths: quick setup, ready-made points, referral, and tier features. **Limitations:** tied to store platforms, subscription cost grows with order volume, and it does not clean or model the merchant's own transaction data.

LoyaltyLion (loyalty platform)

LoyaltyLion offers customizable earning rules, VIP tiers, referral programs, real-time customer lifetime value analytics, and AI-powered campaign recommendations [4]. It integrates with Shopify, BigCommerce, and Magento, and one third-party listing shows a Classic plan at $199 per month including 1,000 orders [9]. Automated daily, weekly, or monthly reports are offered through a separate integration with the Glew analytics platform, available to Glew Plus subscribers [5].

Strengths: rich loyalty rules and a built-in analytics dashboard. **Limitations:** paid subscription, scheduled reporting depends on a third-party add-on, and the data stays inside the vendor's platform.

Shopify Analytics (store reports)

Shopify provides finance, sales, customer, acquisition, and behavior reports, but which reports are available depends on the store's plan, and the custom report builder is limited to the Advanced Shopify and Shopify Plus plans [1].

Strengths: no setup, and the data is already inside the store. **Limitations:** reports cover only data in that platform, there is no built-in loyalty logic, and the merchant has no control over data cleaning or the underlying data model.

Power BI (general business intelligence)

Power BI is a widely used BI tool for dashboards. Microsoft's guidance recommends star schema design so that Power BI models perform well and stay easy to use [6].

Strengths: powerful visualization and filtering. **Limitations:** it assumes data is already cleaned and modeled, it has no loyalty engine, and automatic report generation needs additional setup. It serves as the dashboard layer for our own warehouse.

2.3 Comparison with our project

Criteria	Smile.io	LoyaltyLion	Shopify Analytics	Power BI	Our platform
Loyalty points and tiers	Yes (tiers on paid plans)	Yes	No	No	Yes, editable rules table
Customer segmentation	Limited	Yes	Basic customer reports	Manual	RFM segments
Scheduled weekly and monthly reports	Not a core feature	Via Glew add-on	Manual	Needs extra setup	Built in (Hop and Task Scheduler)
Data cleaning (ETL)	No	No	No	Partial (Power Query)	Yes, Apache Hop
Own relational warehouse	No	No	No	Model only	Yes, PostgreSQL star schema
Platform independence	Store platforms	Store platforms	Shopify only	Any source	Any CSV or database source
Cost	Free to paid plans	Paid plans	Included in Shopify plan	Licence-based	Free tools (Power BI Desktop)
Main limitation	Vendor lock-in	Vendor lock-in	No loyalty logic	No loyalty logic	Prototype scale

2.4 Gap analysis and what we take from the review

- **Gap:** existing loyalty apps focus on shopper-facing rewards, store analytics tools stay inside one platform, and BI tools expect prepared data. None offers cleaning, an owned warehouse, loyalty rules, and automatic reports in one open pipeline.
- From loyalty apps we take simple tier names, points per purchase, and editable rules that business users understand.
- From store analytics we take the core KPIs: revenue, order count, average order value, repeat customers, and top products.
- From BI tools we take date filters, clear layouts, and the star schema design.
- **Our difference:** the full data lifecycle from extraction to scheduled reports, with loyalty rules calculated from the company's own cleaned data.

2.5 Technology options for ETL

The instructor requires an ETL tool or Spark and does not allow Python for ETL or automation. The table compares the options the team considered.

Option	What it is	Fit with our project
Apache Hop	Free, open-source visual ETL tool that runs on Java; pipelines and workflows run from the command line with hop-run [10]	Selected, subject to instructor approval: no Python, free, no licence expiry, and schedulable with Windows Task Scheduler
Talend Studio	Visual ETL tool; the free Talend Open Studio was retired on 31 January 2024, and Studio is now offered as a trial or licensed product [11]	Fallback if the instructor provides the installer
Apache Spark (Spark SQL)	Processing engine; SQL jobs can run from the command line	Fallback: allowed by the instructor, but code-based and less visual
SSIS	Microsoft ETL tool that needs Visual Studio and SQL Server	Not chosen: heavy setup for this project
Power Query	Transformation editor inside Power BI	Used only for light shaping in the dashboard, not as the ETL stage

2.6 Technical foundations

- **Star schema:** dimensional modeling with fact and dimension tables supports fast analytical queries and is recommended for BI models [6][7].
- **RFM analysis:** segmenting customers by recency, frequency, and monetary value is a long-established technique in database marketing [8].
- **ETL with validation:** staged loading, rejected-row logging, and repeatable runs follow standard data warehousing practice [7]. Apache Hop provides these steps visually, and its command-line runner allows scheduled execution [10].

2.7 Feedback & Evaluation

This section records the lecturer's assessment of the project. It is completed by the instructor after reviewing the submission; the team does not write it.

Item	Lecturer's assessment
Overall evaluation of the project	[To be completed by the lecturer]
Strengths noted	[To be completed by the lecturer]
Weaknesses or concerns noted	[To be completed by the lecturer]
Decision on the ETL tool (Apache Hop; see open decisions in 3.6)	[To be completed by the lecturer]
Lecturer name and date	[To be completed by the lecturer]

2.8 Suggested Improvements

Areas where the project can be enhanced. The first group comes from items the team deliberately left out of version 1; the second group comes from the review above.

Area	Suggested improvement	Why it is not in version 1
Loyalty rules interface	A richer interface for editing points and tier rules instead of an editable table or file (risk R6)	Needs instructor input; version 1 keeps a simple rules table
Customer insight	Predictive machine learning, for example churn or next-purchase prediction, on top of RFM segments	Out of scope in 1.1
Data sources	Connect live store APIs instead of a downloaded Kaggle file	Out of scope; only public data is used
Data freshness	Near real-time loading instead of scheduled batches	Real-time streaming is out of scope
Reporting	Report delivery by email or a shared folder, and more report types	Version 1 writes report files and a PDF export from Power BI
Scale	Test beyond 100,000 orders and tune the star schema	Prototype scale (see NFR1)
Loyalty features	Referral programs and rewards redemption, as in Smile.io and LoyaltyLion	Loyalty is limited to points, three tiers, and thresholds (risk R5)

2.9 Final Grading Criteria

The guidelines require a breakdown of marks by documentation, implementation, testing, and presentation. The marks are set by the instructor and have not been shared with the team, so the table below is left for the instructor to complete. The right column shows where the team's evidence for each area is documented.

Area	Marks	Where the team's evidence is
Documentation	[To be set by the instructor]	Parts 1 to 3 (this document), then System Analysis & Design (6 Nov)
Implementation	[To be set by the instructor]	Source code, Hop pipelines, SQL, Power BI dashboard (30 Nov)
Testing	[To be set by the instructor]	Test plan, test cases, KPI checks, bug reports (4 Dec)
Presentation	[To be set by the instructor]	Presentation, user manual, project infographic (4 Dec)

Area	Marks	Where the team's evidence is
Total	[To be set by the instructor]	

2.10 References

[1] Shopify Help Center, "Shopify reports." https://help.shopify.com/manual/reports-and-analytics/shopify-reports (accessed 16 Oct 2026).

[2] Rivo, "Smile.io Review 2026: Features, Pricing & Is It Worth It?" https://www.rivo.io/blog/smile-io-review (accessed 16 Oct 2026). Competitor blog; pricing is indicative.

[3] Capterra, "Smile.io Software Pricing, Alternatives & More 2026." https://www.capterra.com/p/169446/Smile-io/ (accessed 16 Oct 2026).

[4] Capterra, "LoyaltyLion Software Review 2026: Features, Integrations, Pros & Cons." https://www.capterra.com/p/140592/LoyaltyLion/reviews/1175581/ (accessed 16 Oct 2026).

[5] Glew, "LoyaltyLion integration: custom reporting and analytics." https://glew.io/integrations/loyaltylion (accessed 16 Oct 2026).

[6] Microsoft Learn, "Understand star schema and the importance for Power BI." https://learn.microsoft.com/power-bi/guidance/star-schema (accessed 16 Oct 2026).

[7] R. Kimball and M. Ross, The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd ed. Wiley, 2013.

[8] A. M. Hughes, Strategic Database Marketing. Probus Publishing, 1994.

[9] FitGap, "LoyaltyLion: features and pricing." https://us.fitgap.com/products/loyaltylion (accessed 16 Oct 2026). Third-party listing; pricing is indicative.

[10] Apache Hop, "Apache Hop requirements, installation and configuration." https://hop.apache.org/manual/next/installation-configuration.html (accessed 18 Oct 2026).

[11] Qlik Community, reply on the retirement of Talend Open Studio (retired 31 Jan 2024). https://community.qlik.com/t5/Installing-and-Upgrading/Talend-Open-studio-for-big-data-7-3-1/m-p/2458681 (accessed 18 Oct 2026).

Part 3: Requirements Gathering

Requirements describe what the system must do (functional requirements) and how well it must do it (non-functional requirements). We collected them by asking three questions: who will use the platform, what do they need, and what must the project deliver? The table below shows how this part is organized.

Section	Question it answers
3.1 Key terms	What do the technical words mean?
3.2 Stakeholders	Who needs the system?
3.3 User stories and use cases	What do users want to do with it?
3.4 Functional requirements	What must the system do?
3.5 Non-functional requirements	How well must it work?
3.6 Constraints, assumptions, and open decisions	What limits us, and what is still undecided?

3.1 Key terms

Term	Meaning in simple words
ETL	Extract, Transform, Load: collect raw data, clean it, and load it into the database
Pipeline	The automatic chain of steps from raw files to reports
Landing Zone	A folder where raw files are saved exactly as downloaded
Rejected row	A data row that fails a check (for example a negative price); it is saved separately with the reason
Star schema	A database layout with one central table of events (orders) linked to small descriptive tables (customers, products, dates)
KPI	A key number that shows business results, such as revenue or average order value
Loyalty points and tiers	Points customers earn by buying; tiers (Bronze, Silver, Gold) are levels based on total spend
RFM	Grouping customers by how recently, how often, and how much they buy

3.2 Stakeholders

Stakeholders are the people and groups who use the platform or are affected by it.

Stakeholder	How they use the system	What they need
Store owner or manager	Reads dashboards and reports	Clear sales and loyalty results, with reports arriving automatically
Marketing team	Sets loyalty rules, studies customers	Editable points and tiers, top customers, customer groups for campaigns
Data or operations analyst	Runs and checks the pipeline	Trustworthy data, safe reruns, a list of rejected records
IT or system administrator	Schedules and monitors jobs	Logs, scheduled runs, safe handling of passwords
Customers (end shoppers)	Indirectly, through rewards	Fair points and tiers, and protection of their data
Project team	Builds the platform	A clear scope, defined interfaces, tests, and documentation
DEPI instructors and evaluators	Assess the project	Complete documentation, a working demo, and the approved tools

3.3 User stories and use cases

A user story describes one need in the form: as a [user], I want to [action], so that [benefit]. The stories are grouped by user type.

Store manager

ID	I want to...	So that...	Priority
US1	receive weekly and monthly reports automatically	I do not build them by hand	Must
US2	filter the dashboard by date range	I can compare any two periods	Must
US10	export a report as PDF or Excel	I can share it with my team	Should

Marketing manager

ID	I want to...	So that...	Priority
US3	set how many points a purchase earns	rewards match our budget	Must
US4	set the spend thresholds for Bronze, Silver, and Gold	tiers reflect our customer base	Must
US5	see how many customers are in each tier	I can judge the program's reach	Must
US6	group customers by recency, frequency, and spend	I can target campaigns	Should

Data analyst

ID	I want to...	So that...	Priority
US7	see rejected records with the reason	I can fix problems at the source	Must
US8	rerun the pipeline without creating duplicates	corrections are safe	Must

Administrator

ID	I want to...	So that...	Priority
US9	run the pipeline on a schedule with error logs	failures are noticed quickly	Must

Use cases

A use case describes one complete task, step by step. The diagram shows which user or system takes part in which task.

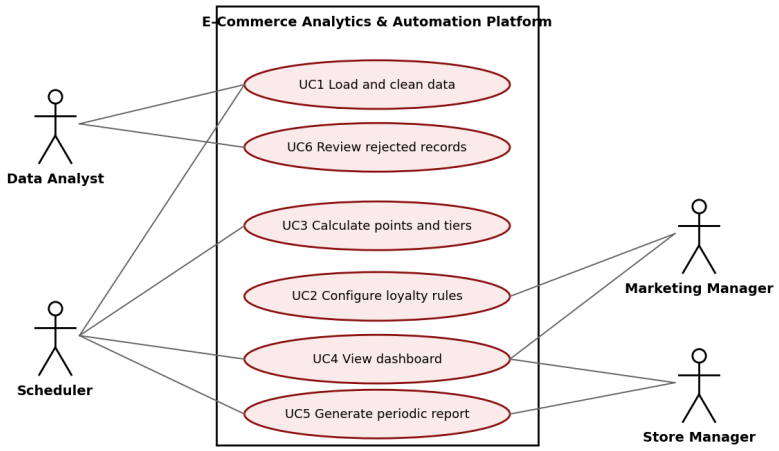


Figure 2. Use case diagram.

ID	Use case	Who	Steps	Result
UC1	Load and clean data	Data analyst, Scheduler	1. Read the Kaggle files in Apache Hop 2. Apply cleaning and validation rules 3. Load valid rows 4. Save rejected rows with a reason	Clean data in the warehouse
UC2	Configure loyalty rules	Marketing manager	1. Open the rules table or file 2. Edit point rate and tier thresholds 3. Save	New rules apply on the next calculation
UC3	Calculate points and tiers	Scheduler (system)	1. Read orders 2. Apply the current rules in SQL 3. Update each customer's points and tier	Customer tiers updated

ID	Use case	Who	Steps	Result
UC4	View dashboard	Store manager, Marketing manager	1. Open the Power BI dashboard 2. Choose a date range 3. View KPIs, trends, and tiers	Insights on sales and loyalty
UC5	Generate periodic report	Scheduler, Store manager	1. At week or month end, the job runs the report queries 2. Report files are written	Report files delivered
UC6	Review rejected records	Data analyst	1. Open the rejected-rows output 2. Filter by reason	List of records to fix

3.4 Functional requirements

Functional requirements say what the system must do. They are grouped by pipeline stage, and each group has one owner. Priority Must means needed for version 1; Should means important but not blocking. Reports use the dataset's own date range because the Kaggle data is historical; the reporting period is chosen with a parameter, not the current date.

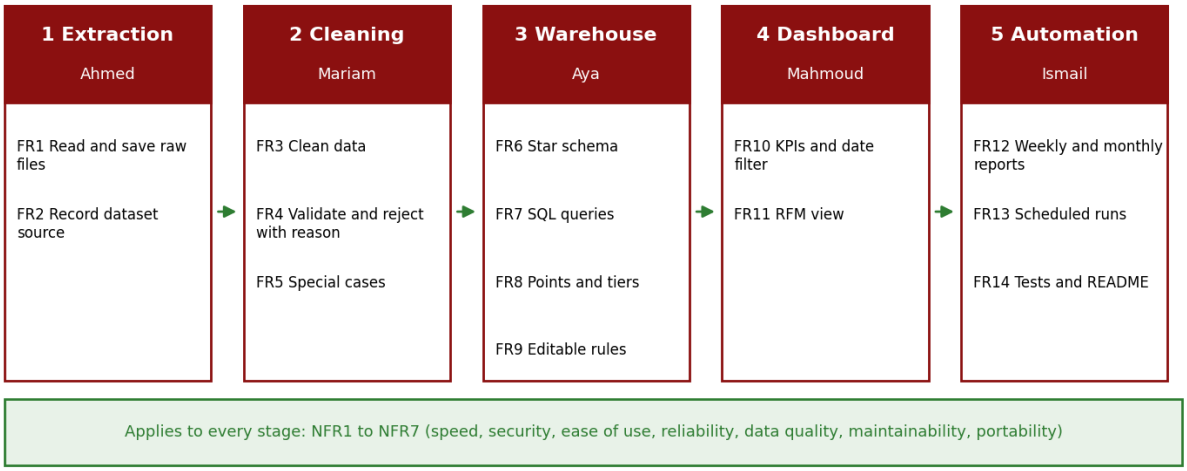


Figure 3. Where each group of requirements sits in the pipeline.

A. Extraction (owner: Ahmed)

ID	Requirement	Priority
FR1	Read the chosen Kaggle e-commerce CSV files in Apache Hop, keep unchanged raw copies in a Landing Zone, and load them into staging	Must
FR2	Record the dataset name, link, licence, and download date in the README	Must

B. Cleaning (owner: Mariam)

ID	Requirement	Priority
FR3	Clean data in Apache Hop: handle missing values and duplicates; standardize types and dates	Must
FR4	Validate records (no negative prices, unique IDs, required fields) and write rejected rows with a reason	Must
FR5	Apply a documented rule to cancelled orders and negative quantities (reject, or move to a returns table)	Should

C. Warehouse and loyalty rules (owner: Aya)

ID	Requirement	Priority
FR6	Store clean data in a PostgreSQL star schema with keys and relationships; reruns update existing rows without duplicates	Must
FR7	Provide SQL queries for revenue, top products, top categories, and orders per period	Must
FR8	Calculate loyalty points and tiers per customer in SQL from a rules table	Must
FR9	Keep loyalty rules (point rate, tier thresholds) in an editable table or file; the pipeline recalculates on the next run (with Mahmoud)	Must

D. Dashboard (owner: Mahmoud)

ID	Requirement	Priority
FR10	Show KPIs, trends, and tier distribution in Power BI with a date filter, using DAX measures	Must
FR11	Provide an RFM customer segmentation view	Should

E. Automation and reports (owner: Ismail)

ID	Requirement	Priority
FR12	Generate weekly and monthly report files (Excel or CSV) from the scheduled pipeline, and provide a Power BI report page that exports to PDF (with Mahmoud)	Must
FR13	Run the full pipeline on a schedule with Windows Task Scheduler and Hop's command-line runner, in the correct order, with error handling and logs	Must
FR14	Provide tests for each stage and a README with setup steps, including Java and Hop versions (with Ahmed)	Should

3.5 Non-functional requirements

Non-functional requirements say how well the system must work. Each one has a way to check it.

ID	Quality	In simple words	How we check it
NFR1	Speed	Screens open and reports finish quickly, on a sample of up to 100,000 orders	Dashboard view under 5 s; monthly report under 5 min
NFR2	Security and privacy	Only public data is used, and no passwords are stored in GitHub	Repository review finds no secrets
NFR3	Ease of use	A non-technical user can change loyalty rules and open reports without code	At least 80% of 5 test users succeed unaided
NFR4	Reliability	A failed run does not damage data, and a rerun does not create duplicates	Failure and rerun tests pass
NFR5	Data quality	Bad data is rejected with a reason, never silently changed to zero	At least 95% of sample rows pass; all rejections logged
NFR6	Maintainability	Pipelines, SQL scripts, and a README are stored in Git	Each stage runs and is documented on its own
NFR7	Portability	Runs on Windows with Java, Apache Hop, PostgreSQL, and Power BI Desktop	A clean setup works from the README

3.6 Constraints, assumptions, and open decisions

Constraints

- The instructor does not allow Python for ETL or automation. ETL must use an ETL tool or Spark.
- **The documentation deadline is Monday, 19 October 2026 and is final (no extension).** The rest of the project follows the DEPI schedule, with final delivery on 4 December 2026.
- Only free or provided tools and public data are used.

Assumptions

- The Kaggle dataset has customer, product, and order identifiers that link its tables.

- The dataset is downloaded manually, because the Kaggle command-line tool is built on Python.

Open decisions

Decision	Options	Who decides	Target date
ETL tool	Apache Hop; fallbacks are Spark SQL, or Talend Studio if the instructor provides the installer	Instructor (Ahmed asks)	30 Oct 2026
Database	PostgreSQL (team proposal) or MySQL	Aya with the team	6 Nov 2026
Kaggle dataset	Olist Brazilian E-Commerce or Online Retail II	Whole team	6 Nov 2026
Loyalty rule editing	A rules table or file, or a richer interface	Instructor and Mahmoud	6 Nov 2026